

CMU · 15-721 | Advanced Database Systems (2020)

15-721 (2021) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1wv411w7Ko>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-15-721>

内存数据库

并发控制

数据库压缩

哈希连接

超内存数据库

存储模型

调度规划

查询执行

索引

协议

网络

查询编译

查询优化

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程资料包页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击底部菜单栏
称为 AI 内容创作者？回复 [添砖加瓦]

Lecture #24

Carnegie Mellon University

ADVANCED DATABASE SYSTEMS

Server-side Logic Execution

@Andy_Pavlo // 15-721 // Spring 2020

TODAY'S AGENDA

Background

UDF In-lining

UDF CTE Conversion



OBSERVATION

Until now, we have assumed that all of the logic for an application is located in the application itself.

The application has a "conversation" with the DBMS to store/retrieve data.

→ Protocols: JDBC, ODBC



CONVERSATIONAL DATABASE API

Application



BEGIN

SQL

Program Logic

SQL

Program Logic

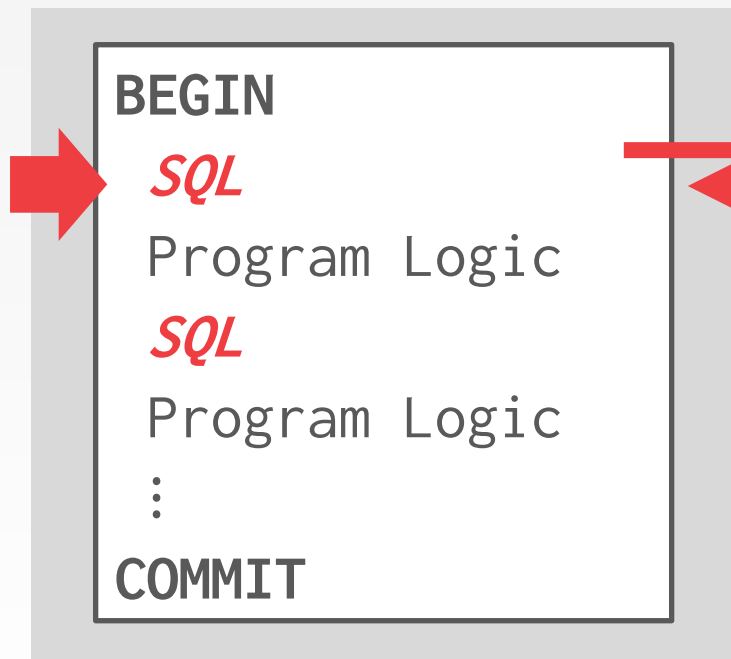
⋮

COMMIT



CONVERSATIONAL DATABASE API

Application

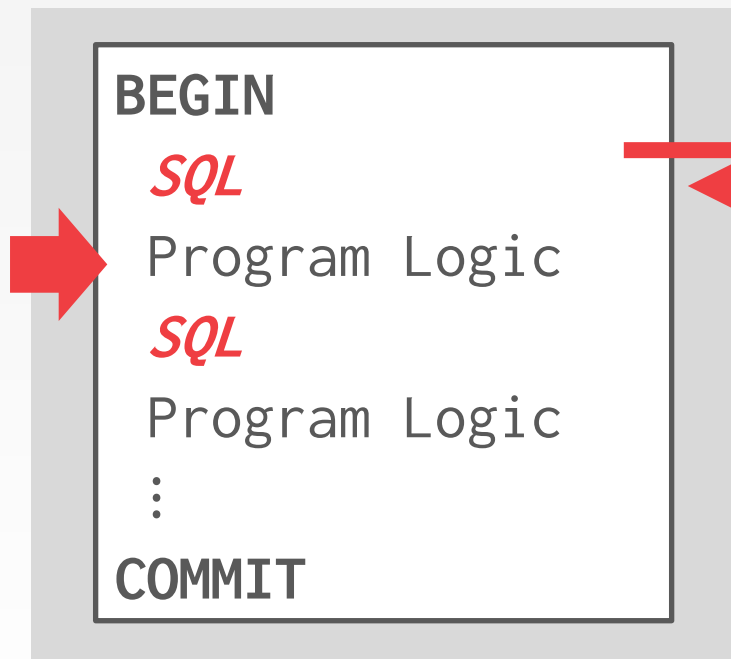


Parser
Planner
Optimizer
Query Execution



CONVERSATIONAL DATABASE API

Application

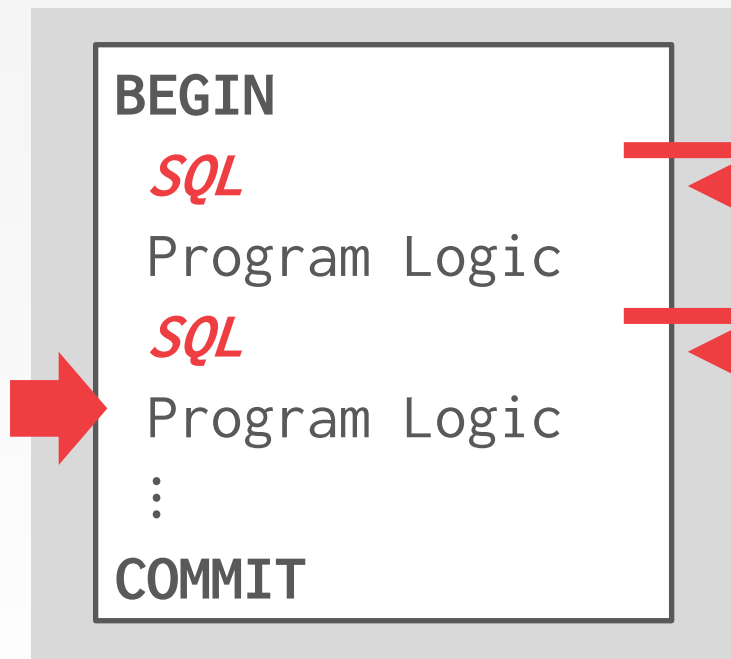


*Parser
Planner
Optimizer
Query Execution*



CONVERSATIONAL DATABASE API

Application

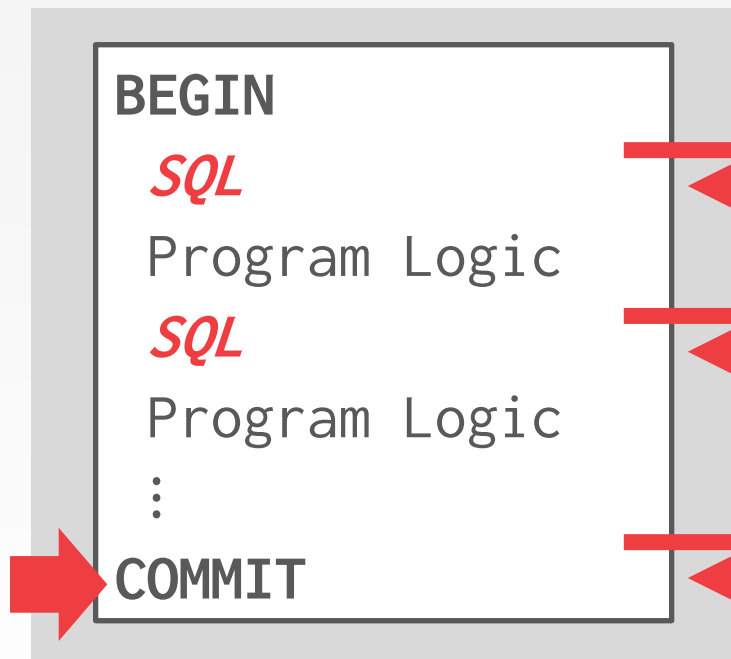


*Parser
Planner
Optimizer
Query Execution*



CONVERSATIONAL DATABASE API

Application



*Parser
Planner
Optimizer
Query Execution*



EMBEDDED DATABASE LOGIC

Move application logic into the DBMS to avoid multiple network round-trips and to extend the functionality of the DBMS.

Potential Benefits

- Efficiency
- Reuse



EMBEDDED DATABASE LOGIC

Application

```
BEGIN
```

```
SQL
```

```
Program Logic
```

```
SQL
```

```
Program Logic
```

```
⋮
```

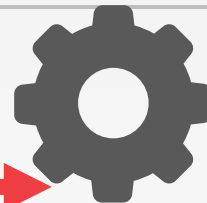
```
COMMIT
```



EMBEDDED DATABASE LOGIC

Application

CALL PROC(x=99)



PROC(x)

```
BEGIN
  SQL
  Program Logic
  SQL
  Program Logic
  :
COMMIT
```



EMBEDDED DATABASE LOGIC

User-Defined Functions (UDFs)

Stored Procedures

Triggers

User-Defined Types (UDTs)

User-Defined Aggregates (UDAs)



USER-DEFINED FUNCTIONS

A **user-defined function** (UDF) is a function written by the application developer that extends the system's functionality beyond its built-in operations.

- It takes in input arguments (scalars)
- Perform some computation
- Return a result (scalars, tables)



UDF EXAMPLE

```
CREATE FUNCTION cust_level(@ckey int)
RETURNS char(10) AS
BEGIN
  DECLARE @total float;
  DECLARE @level char(10);

  SELECT @total = SUM(o_totalprice)
    FROM orders WHERE o_custkey=@ckey;

  IF (@total > 1000000)
    SET @level = 'Platinum';
  ELSE
    SET @level = 'Regular';

  RETURN @level;
END
```

Get all the customer ids and compute their customer service level based on the amount of money they have spent.

```
SELECT c_custkey,
       cust_level(c_custkey)
FROM customer
```

UDF ADVANTAGES

They encourage modularity and code reuse

→ Different queries can reuse the same application logic without having to reimplement it each time.

Fewer network round-trips between application server and DBMS for complex operations.

Some types of application logic are easier to express and read as UDFs than SQL.

UDF DISADVANTAGES (1)

Query optimizers treat UDFs as black boxes.

→ Unable to estimate cost if you don't know what a UDF is going to do when you run it.

It is difficult to parallelize UDFs due to correlated queries inside of them.

- Some DBMSs will only execute queries with a single thread if they contain a UDF.
- Some UDFs incrementally construct queries.

UDF DISADVANTAGES (2)

Complex UDFs in **SELECT** / **WHERE** clauses force the DBMS to execute iteratively.

- RBAR = "Row By Agonizing Row"
- Things get even worse if UDF invokes queries due to implicit joins that the optimizer cannot "see".

Since the DBMS executes the commands in the UDF one-by-one, it is unable to perform cross-statement optimizations.

UDF PERFORMANCE

Microsoft SQL Server

TPC-H Q12 using a UDF (SF=1).

```
SELECT l_shipmode,  
       SUM(CASE  
           WHEN o_orderpriority <> '1-URGENT'  
           THEN 1 ELSE 0 END  
       ) AS low_line_count  
FROM   orders, lineitem  
WHERE  o_orderkey = l_orderkey  
       AND l_shipmode IN ('MAIL','SHIP')  
       AND l_commitdate < l_receiptdate  
       AND l_shipdate < l_commitdate  
       AND l_receiptdate >= '1994-01-01'  
       AND dbo.cust_name(o_custkey) IS NOT NULL  
GROUP BY l_shipmode  
ORDER BY l_shipmode
```

Source: [Karthik Ramachandra](#)

UDF PERFORMANCE

Microsoft SQL Server

TPC-H Q12 using a UDF (SF=1).

→ **Original Query:** 0.8 sec

→ **Query + UDF:** 13 hr 30 min

```
SELECT l_shipmode,
       SUM(CASE
            WHEN o_orderpriority <> '1-URGENT'
            THEN 1 ELSE 0 END
       ) AS low_line_count
FROM orders, lineitem
WHERE o_orderkey = l_orderkey
      AND l_shipmode IN ('MAIL', 'SHIP')
      AND l_commitdate < l_receiptdate
      AND l_shipdate < l_commitdate
      AND l_receiptdate >= '1994-01-01'
      AND dbo.cust_name(o_custkey) IS NOT NULL
GROUP BY l_shipmode
ORDER BY l_shipmode
```

```
CREATE FUNCTION cust_name(@ckey int)
RETURNS char(25) AS
BEGIN
  DECLARE @n char(25);
  SELECT @n = c_name
    FROM customer WHERE c_custkey = @ckey;
  RETURN @n;
END
```

MICROSOFT SQL SERVER UDF HISTORY

2001 – Microsoft adds TSQL Scalar UDFs.

2008 – People realize that UDFs are "evil".

Source: [Karthik Ramachandra](#)



TSQL Scalar functions are evil.

I've been working with a number of clients recently who all have suffered at the hands of TSQL Scalar functions. Scalar functions were introduced in SQL 2000 as a means to wrap logic so we benefit from code reuse and simplify our queries. Who would be daft enough not to think this was a good idea. I for one jumped on this initially thinking it was a great thing to do.

However as you might have gathered from the title scalar functions aren't the nice friend you may think they are.

If you are running queries across large tables then this may explain why you are getting poor performance.

In this post we will look at a simple padding function, we will be creating large volumes to emphasize the issue with scalar udfs.

```
create function PadLeft(@val varchar(100), @len int, @char char(1))
returns varchar(100)
as
begin
    return right(replicate(@char,@len) + @val, @len)
end
go
```

Interpreted

Scalar functions are interpreted code that means EVERY call to the function results in your code being interpreted. That means overhead for processing your function is proportional to the number of rows.

Running this code you will see that the native system calls take considerable less time than the UDF calls. On my machine it takes 2614 ms for the system calls and 38758ms for the UDF. That's a 19x increase.

```
set statistics time on
go
select max(right(replicate('0',100) + o.name + c.name, 100))
from msdb.sys.columns o
cross join msdb.sys.columns c

select max(dbo.PadLeft(o.name + c.name, 100, '0'))
from msdb.sys.columns o
cross join msdb.sys.columns c
```

Source:

OF HISTORY

UDFs.

evil".

MICROSOFT SQL SERVER UDF HISTORY

2001 – Microsoft adds TSQL Scalar UDFs.

2008 – People realize that UDFs are "evil".

2010 – Microsoft acknowledges that UDFs are evil.

Source: [Karthik Ramachandra](#)

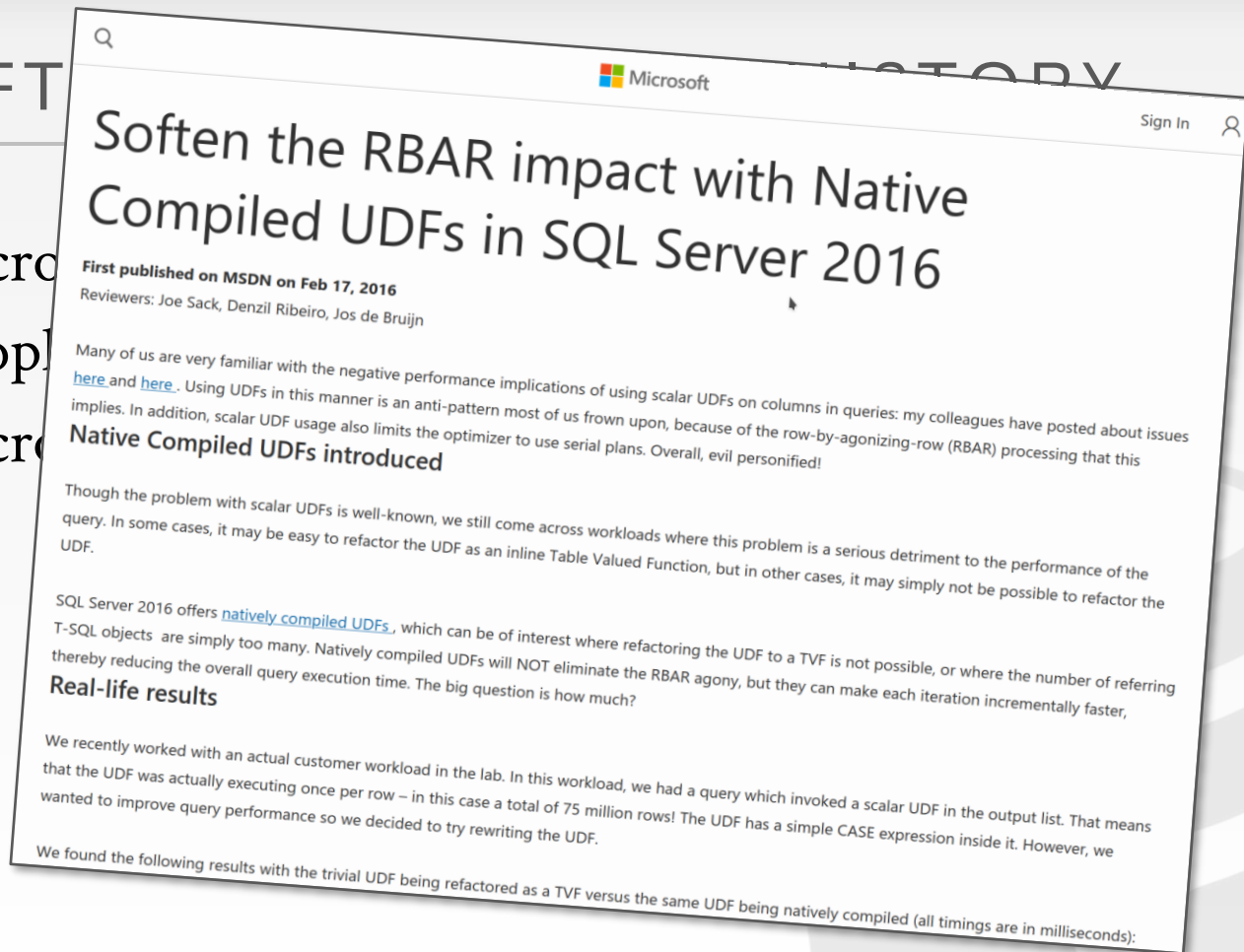


MICROSOFT

2001 – Micro

2008 – People

2010 – Micro



Source: [Karthik Ramachandra](#)

MICROSOFT SQL SERVER UDF HISTORY

2001 – Microsoft adds TSQL Scalar UDFs.

2008 – People realize that UDFs are "evil".

2010 – Microsoft acknowledges that UDFs are evil.

2014 – UDF decorrelation research @ IIT-B.

2015 – Froid project begins @ MSFT Gray Lab.

2018 – Froid added to SQL Server 2019.

The screenshot shows the Microsoft SQL Docs website. The top navigation bar includes the Microsoft logo, 'SQL Docs', and various menu items like 'Overview', 'Install', 'Secure', 'Develop', 'Administer', and 'More'. A 'Download SQL Server' button is also present. The breadcrumb trail indicates the path: 'Docs / SQL / Database design / User-defined functions / Scalar inlining'. On the right, there are links for 'Edit', 'Share', 'Dark' theme, and a user profile 'assface'.

The main content area is titled 'Scalar UDF Inlining' with a subtitle '02/27/2019 • 10 minutes to read • Contributors'. Below this, the 'APPLIES TO:' section shows that the article applies to 'SQL Server' and 'Azure SQL Database' (both with green checkmarks) and does not apply to 'Azure SQL Data Warehouse' (marked with a red X). The text states: 'This article introduces Scalar UDF inlining, a feature under the intelligent query processing suite of features. This feature improves the performance of queries that invoke scalar UDFs in SQL Server (starting with SQL Server 2019 preview) and SQL Database.'

On the left, a sidebar shows a list of topics under 'Azure SQL Database - current'. The 'Scalar inlining' topic is highlighted in blue. Other topics include 'Nondeterministic Functions', 'Create', 'Modify', 'Delete', 'Execute', 'Rename', 'View', 'Views', 'Development', 'Internals & Architecture', 'Installation', and 'Download PDF'.

Below the main text, there is a section titled 'T-SQL Scalar User-Defined Functions' which explains that User-Defined Functions (UDFs) are implemented in Transact-SQL and return a single data value. It notes that T-SQL Scalar UDFs are an elegant way to achieve code reuse and modularity across SQL queries, but some computations (like complex business rules) are easier to express in imperative UDF form. UDFs help in building up complex logic without requiring expertise in writing complex SQL queries.

At the bottom, a section titled 'Performance of Scalar UDFs' begins with the text: 'Scalar UDFs typically end up performing poorly due to the following reasons.'

On the right side, under the heading 'In this article', there is a list of links: 'T-SQL Scalar User-Defined Functions', 'Performance of Scalar UDFs', 'Automatic Inlining of Scalar UDFs', 'Inlineable Scalar UDFs requirements', 'Enabling scalar UDF inlining', 'Disabling Scalar UDF inlining without changing the compatibility level', 'Important Notes', and 'See Also'.

Source: [Karthik](#)

FROID

Automatically convert UDFs into relational expressions that are inlined as sub-queries.

→ Does not require the app developer to change UDF code.

Perform conversion during the rewrite phase to avoid having to change the cost-base optimizer.

→ Commercial DBMSs already have powerful transformation rules for executing sub-queries efficiently.

SUB-QUERIES

The DBMS treats nested sub-queries in the where clause as functions that take parameters and return a single value or set of values.

Two Approaches:

- Rewrite to de-correlate and/or flatten them
- Decompose nested query and store result to temporary table. Then the outer joins with the temporary table.

SUB-QUERIES – REWRITE

```
SELECT name FROM sailors AS S
WHERE EXISTS (
  SELECT * FROM reserves AS R
  WHERE S.sid = R.sid
  AND R.day = '2020-04-22'
)
```



```
SELECT name
FROM sailors AS S, reserves AS R
WHERE S.sid = R.sid
AND R.day = '2020-04-22'
```

LATERAL JOIN

A lateral inner subquery can refer to fields in rows of the table reference to determine which rows to return.

→ Allows you to have sub-queries in **FROM** clause.

The DBMS iterates through each row in the table referenced and evaluates the inner sub-query for each row.

→ The rows returned by the inner sub-query are added to the result of the join with the outer query.

FROID OVERVIEW

Step #1 – Transform Statements

Step #2 – Break UDF into Regions

Step #3 – Merge Expressions

Step #4 – Inline UDF Expression into Query

Step #5 – Run Through Query Optimizer

STEP #1 – TRANSFORM STATEMENTS

Imperative Statements

```
SET @level = 'Platinum';
```

```
SELECT @total = SUM(o_totalprice)
FROM orders
WHERE o_custkey=@ckey;
```

```
IF (@total > 1000000)
  SET @level = 'Platinum';
```

SQL Statements

```
SELECT 'Platinum' AS level;
```

```
SELECT (
  SELECT SUM(o_totalprice)
  FROM orders
  WHERE o_custkey=@ckey
) AS total;
```

```
SELECT (
  CASE WHEN total > 1000000
  THEN 'Platinum'
  ELSE NULL
END) AS level;
```


STEP #2 – BREAK INTO REGIONS

```
CREATE FUNCTION cust_level(@ckey int)
RETURNS char(10) AS
BEGIN
  DECLARE @total float;
  DECLARE @level char(10);

  SELECT @total = SUM(o_totalprice)
    FROM orders WHERE o_custkey=@ckey;

  IF (@total > 1000000)
    SET @level = 'Platinum';
  ELSE
    SET @level = 'Regular';

  RETURN @level;
END
```



STEP #2 – BREAK INTO REGIONS

```
CREATE FUNCTION cust_level(@ckey int)
RETURNS char(10) AS
BEGIN
```

1

```
DECLARE @total float;
DECLARE @level char(10);

SELECT @total = SUM(o_totalprice)
FROM orders WHERE o_custkey=@ckey;
```

```
IF (@total > 1000000)
    SET @level = 'Platinum';
ELSE
    SET @level = 'Regular';

RETURN @level;
END
```

```
(SELECT NULL AS level,
 (SELECT SUM(o_totalprice)
  FROM orders
  WHERE o_custkey=@ckey) AS total
) AS E_R1
```

STEP #2 – BREAK INTO REGIONS

```
CREATE FUNCTION cust_level(@ckey int)
RETURNS char(10) AS
BEGIN
```

1

```
DECLARE @total float;
DECLARE @level char(10);

SELECT @total = SUM(o_totalprice)
FROM orders WHERE o_custkey=@ckey;
```

2

```
IF (@total > 1000000)
SET @level = 'Platinum';
ELSE
SET @level = 'Regular';

RETURN @level;
END
```

```
(SELECT NULL AS level,
(SELECT SUM(o_totalprice)
FROM orders
WHERE o_custkey=@ckey) AS total
) AS E_R1
```

```
(SELECT (
CASE WHEN E_R1.total > 1000000
THEN 'Platinum'
ELSE E_R1.level END) AS level
) AS E_R2
```

STEP #2 – BREAK INTO REGIONS

```
CREATE FUNCTION cust_level(@ckey int)
RETURNS char(10) AS
BEGIN
```

1

```
DECLARE @total float;
DECLARE @level char(10);

SELECT @total = SUM(o_totalprice)
FROM orders WHERE o_custkey=@ckey;
```

2

```
IF (@total > 1000000)
SET @level = 'Platinum';
```

3

```
ELSE
SET @level = 'Regular';
```

```
RETURN @level;
END
```

```
(SELECT NULL AS level,
 (SELECT SUM(o_totalprice)
  FROM orders
   WHERE o_custkey=@ckey) AS total
) AS E_R1
```

```
(SELECT (
 CASE WHEN E_R1.total > 1000000
 THEN 'Platinum'
 ELSE E_R1.level END) AS level
) AS E_R2
```

```
(SELECT (
 CASE WHEN E_R1.total <= 1000000
 THEN 'Regular'
 ELSE E_R2.level END) AS level
) AS E_R3
```

STEP #2 – BREAK INTO REGIONS

```
CREATE FUNCTION cust_level(@ckey int)
RETURNS char(10) AS
BEGIN
```

1 **DECLARE** @total float;
DECLARE @level char(10);

SELECT @total = **SUM**(o_totalprice)
FROM orders **WHERE** o_custkey=@ckey;

2 **IF** (@total > 1000000)
SET @level = '**Platinum**';

3 **ELSE**
SET @level = '**Regular**';

4 **RETURN** @level;

```
END
```

```
(SELECT NULL AS level,  
  (SELECT SUM(o_totalprice)  
    FROM orders  
    WHERE o_custkey=@ckey) AS total  
  ) AS E_R1
```

```
(SELECT (  
  CASE WHEN E_R1.total > 1000000  
  THEN 'Platinum'  
  ELSE E_R1.level END) AS level  
) AS E_R2
```

```
(SELECT (  
  CASE WHEN E_R1.total <= 1000000  
  THEN 'Regular'  
  ELSE E_R2.level END) AS level  
) AS E_R3
```

STEP #3 – MERGE EXPRESSIONS

```
(SELECT NULL AS level,
 (SELECT SUM(o_totalprice)
  FROM orders
  WHERE o_custkey=@ckey) AS total
) AS E_R1
```



```
(SELECT (
 CASE WHEN E_R1.total > 1000000
 THEN 'Platinum'
 ELSE E_R1.level END) AS level
) AS E_R2
```



```
(SELECT (
 CASE WHEN E_R1.total <= 1000000
 THEN 'Regular'
 ELSE E_R2.level END) AS level
) AS E_R3
```



```
SELECT E_R3.level FROM
 (SELECT NULL AS level,
  (SELECT SUM(o_totalprice)
   FROM orders
   WHERE o_custkey=@ckey) AS total
 ) AS E_R1
CROSS APPLY
 (SELECT (
  CASE WHEN E_R1.total > 1000000
  THEN 'Platinum'
  ELSE E_R1.level END) AS level
 ) AS E_R2
CROSS APPLY
 (SELECT (
  CASE WHEN E_R1.total <= 1000000
  THEN 'Regular'
  ELSE E_R2.level END) AS level
 ) AS E_R3;
```

STEP #4 – INLINE EXPRESSION

Original Query

```
SELECT c_custkey,  
       cust_level(c_custkey)  
FROM customer
```



STEP #4 – INLINE EXPRESSION

Original Query

```
SELECT c_custkey,
       cust_level(c_custkey)
FROM customer
```



```
SELECT c_custkey, (
  SELECT E_R3.level FROM
    (SELECT NULL AS level,
      (SELECT SUM(o_totalprice)
       FROM orders
        WHERE o_custkey=@ckey) AS total
     ) AS E_R1
  CROSS APPLY
    (SELECT (
      CASE WHEN E_R1.total > 1000000
      THEN 'Platinum'
      ELSE E_R1.level END) AS level
     ) AS E_R2
  CROSS APPLY
    (SELECT (
      CASE WHEN E_R1.total <= 1000000
      THEN 'Regular'
      ELSE E_R2.level END) AS level
     ) AS E_R3;
) FROM customer;
```


STEP #4 – INLINE EXPRESSION

Original Query

```
SELECT c_custkey,
       cust_level(c_custkey)
FROM customer
```



```
SELECT c_custkey, (
  4 SELECT E_R3.level FROM
    1 (SELECT NULL AS level,
      (SELECT SUM(o_totalprice)
       FROM orders
       WHERE o_custkey=@ckey) AS total
     ) AS E_R1
    CROSS APPLY
    2 (SELECT (
      CASE WHEN E_R1.total > 1000000
      THEN 'Platinum'
      ELSE E_R1.level END) AS level
    ) AS E_R2
    CROSS APPLY
    3 (SELECT (
      CASE WHEN E_R1.total <= 1000000
      THEN 'Regular'
      ELSE E_R2.level END) AS level
    ) AS E_R3;
  ) FROM customer;
```

STEP #5 - OPTIMIZE

```
SELECT c_custkey, (  
  SELECT E_R3.level FROM  
    (SELECT NULL AS level,  
      (SELECT SUM(o_totalprice)  
        FROM orders  
        WHERE o_custkey=@ckey) AS total  
    ) AS E_R1  
  CROSS APPLY  
    (SELECT (  
      CASE WHEN E_R1.total > 1000000  
        THEN 'Platinum'  
      ELSE E_R1.level END) AS level  
    ) AS E_R2  
  CROSS APPLY  
    (SELECT (  
      CASE WHEN E_R1.total <= 1000000  
        THEN 'Regular'  
      ELSE E_R2.level END) AS level  
    ) AS E_R3;  
) FROM customer;
```

STEP #5 - OPTIMIZE

```

SELECT c_custkey, (
  SELECT E_R3.level FROM
    (SELECT NULL AS level,
      (SELECT SUM(o_totalprice)
       FROM orders
        WHERE o_custkey=@ckey) AS total
     ) AS E_R1
  CROSS APPLY
    (SELECT (
      CASE WHEN E_R1.total > 1000000
      THEN 'Platinum'
      ELSE E_R1.level END) AS level
     ) AS E_R2
  CROSS APPLY
    (SELECT (
      CASE WHEN E_R1.total <= 1000000
      THEN 'Regular'
      ELSE E_R2.level END) AS level
     ) AS E_R3;
) FROM customer;

```



```

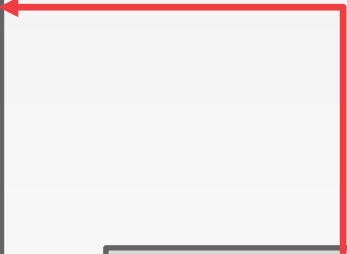
SELECT c.c_custkey,
       CASE WHEN e.total > 1000000
       THEN 'Platinum'
       ELSE 'Regular'
       END
FROM customer c LEFT OUTER JOIN
  (SELECT o_custkey,
    SUM(o_totalprice) AS total
   FROM order GROUP BY o_custkey
  ) AS e
ON c.c_custkey=e.o_custkey;

```

BONUS OPTIMIZATIONS

```
CREATE FUNCTION getVal(@x int)
RETURNS char(10) AS
BEGIN
  DECLARE @val char(10);
  IF (@x > 1000)
    SET @val = 'high';
  ELSE
    SET @val = 'low';
  RETURN @val + ' value';
END
```

```
SELECT getVal(5000);
```



BONUS OPTIMIZATIONS

```
CREATE FUNCTION getVal(@x int)
RETURNS char(10) AS
BEGIN
  DECLARE @val char(10);
  IF (@x > 1000)
    SET @val = 'high';
  ELSE
    SET @val = 'low';
  RETURN @val + ' value';
END
```

Froid



```
SELECT returnVal FROM
  (SELECT CASE WHEN @x > 1000
    THEN 'high'
    ELSE 'low' END AS val)
  AS DT1
OUTER APPLY
  (SELECT DT1.val + ' value'
    AS returnVal) DT2
```

BONUS OPTIMIZATIONS

```
CREATE FUNCTION getVal(@x int)
RETURNS char(10) AS
BEGIN
  DECLARE @val char(10);
  IF (@x > 1000)
    SET @val = 'high';
  ELSE
    SET @val = 'low';
  RETURN @val + ' value';
END
```

Froid



```
SELECT returnVal FROM
  (SELECT CASE WHEN @x > 1000
    THEN 'high'
    ELSE 'low' END AS val)
  AS DT1
OUTER APPLY
  (SELECT DT1.val + ' value'
    AS returnVal) DT2
```

```
BEGIN
  DECLARE @val char(10);
  SET @val = 'high';
  RETURN @val + ' value';
END
```

Dynamic Slicing

```
SELECT returnVal FROM
  (SELECT 'high' AS val)
  AS DT1
OUTER APPLY
  (SELECT DT1.val + ' value'
    AS returnVal)
  AS DT2
```

```
BEGIN
  DECLARE @val char(10);
  SET @val = 'high';
  RETURN 'high value';
END
```

Const Propagation & Folding

```
SELECT returnVal FROM
  (SELECT 'high value'
    AS returnVal)
  AS DT1
```

```
BEGIN
  RETURN 'high value';
END
```

Dead Code Elimination

```
SELECT 'high value';
```

SUPPORTED OPERATIONS (2019)

T-SQL Syntax:

- **DECLARE**, **SET** (variable declaration, assignment)
- **SELECT** (SQL query, assignment)
- **IF** / **ELSE** / **ELSE IF** (arbitrary nesting)
- **RETURN** (multiple occurrences)
- **EXISTS**, **NOT EXISTS**, **ISNULL**, **IN**, ... (Other relational algebra operations)

UDF invocation (nested/recursive with configurable depth)

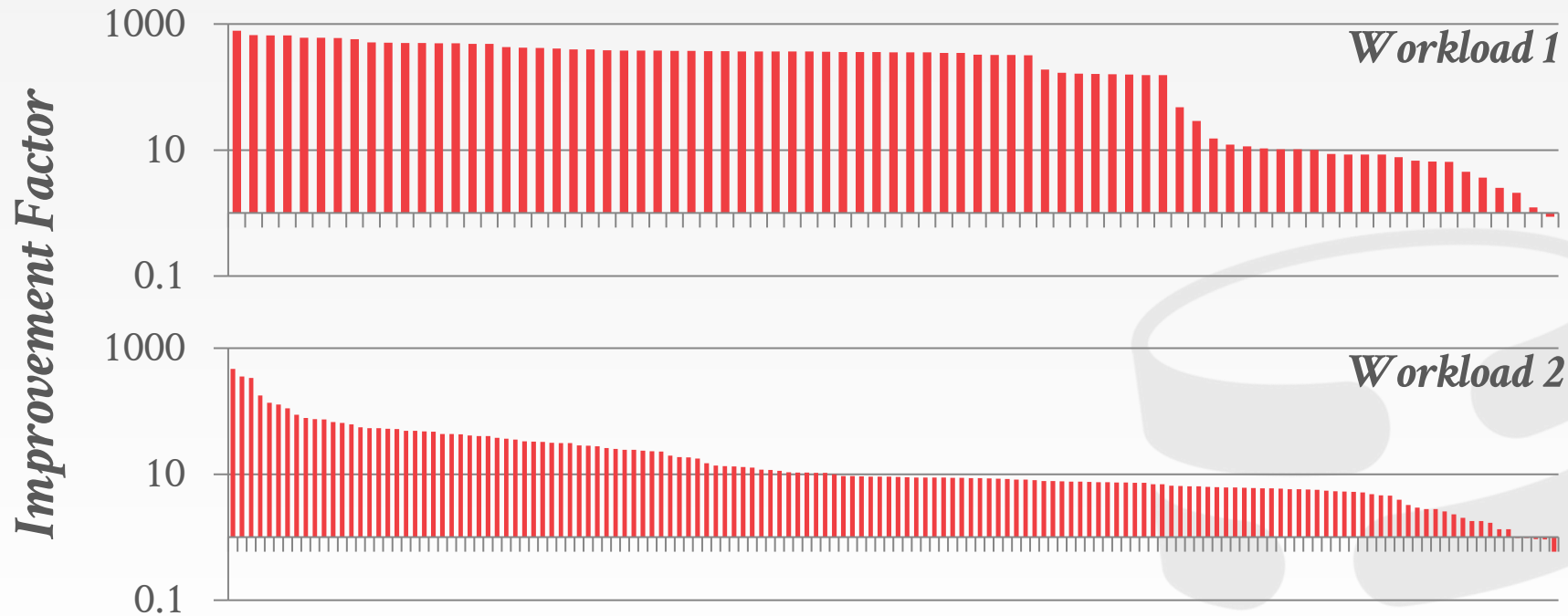
All SQL datatypes.

APPLICABILITY / COVERAGE

	<i># of Scalar UDFs</i>	<i>Froid Compatible</i>	
Workload 1	178	150	84%
Workload 2	90	82	91%
Workload 3	22	21	95%

UDF IMPROVEMENT STUDY

Table: 100k Tuples



Source: [Karthik Ramachandra](#)

UDFs-TO-CTEs

Rewrite UDFs into plain SQL commands.

Use recursive common table expressions (CTEs) to support iterations and other control flow concepts not supported in Froid.

DBMS Agnostic

→ Can be implemented as a rewrite middleware layer on top of any DBMS that supports CTEs.



UDFs-TO-CTEs OVERVIEW

- Step #1 – Static Single Assignment Form
- Step #2 – Administrative Normal Form
- Step #3 – Mutual to Direct Recursion
- Step #4 – Tail Recursion to **WITH RECURSIVE**
- Step #5 – Run Through Query Optimizer

STEP #1 – STATIC SINGLE ASSIGNMENT

```

CREATE FUNCTION pow(x int, n int)
RETURNS int AS
$$
  DECLARE
    i int = 0;
    p int = 1;
  BEGIN
    WHILE i < n LOOP
      p = p * x;
      i = i + 1;
    END LOOP;
    RETURN p;
  END;
$$

```



```

pow(x,n):
  i0 ← 0;
  p0 ← 1;
  while: i1 ← Φ(i0, i2);
        p1 ← Φ(p0, p2);
        if i1 < n then
          goto loop;
        else
          goto exit;
  loop: p2 ← p1 * x;
        i2 ← i1 + 1;
        goto while;
  exit: return p1;

```

STEP #2 – ADMINISTRATIVE NORMAL FORM

```

pow(x,n):
    i0 ← 0;
    p0 ← 0;
    while: i1 ← Φ(i0,i2);
           p1 ← Φ(p0,p2);
           if i1 < n then
               goto loop;
           else
               goto exit;
    loop: p2 ← p1 * x;
           i2 ← i1 + 1;
           goto while;
    exit: return p1;
  
```



```

pow(x,n) =
    let i0 = 0 in
    let p0 = 1 in
        while(i0,p0,x,n)

    while(i1,p1,x,n) =
        let t0 = i1 >= n in
        if t0 then p1
        else body(i1,p1,x,n)

    body(i1,p1,x,n) =
        let p2 = p1 * x in
        let i2 = i1 + 1 in
            while(i2,p2,x,n)
  
```

STEP #3 – MUTUAL TO DIRECT RECURSION

```

pow(x,n) =
  let i0 = 0 in
    let p0 = 1 in
      while(i0,p0,x,n)

while(i1,p1,x,n) =
  let t0 = i1 >= n in
    if t0 then p1
    else body(i1,p1,x,n)

body(i1,p1,x,n) =
  let p2 = p1 * x in
    let i2 = i1 + 1 in
      while(i2,p2,x,n)
  
```



```

pow(x,n) =
  let i0 = 0 in
    let p0 = 1 in
      run(i0,p0,x,n)

run(i1,p1,x,n) =
  let t0 = i1 >= n in
    if t0 then p1
    else
      let p2 = p1 * x in
        let i2 = i1 + 1 in
          run(i2,p2,x,n)
  
```

STEP #4 – WITH RECURSIVE

```

pow(x,n) =
  let i0 = 0 in
  let p0 = 1 in
    run(i0,p0,x,n)

run(i1,p1,x,n) =
  let t0 = i1 >= n in
  if t0 then p1
  else
    let p2 = p1 * x in
    let i2 = i1 + 1 in
      run(i2,p2,x,n)
  
```



```

WITH RECURSIVE
  run("call?",i1,p1,x,n,result) AS (
    SELECT true,0,1,x,n,NULL

    UNION ALL
    SELECT iter.* FROM run, LATERAL (
      SELECT false,0,0,0,0,p1
      WHERE i1 >= n
      UNION ALL
      SELECT true,i1+1,p1*x,x,n,0
      WHERE i1 < n
    ) AS iter("call?",i1,p1,x,n,result)
    WHERE run."call?"
  )
SELECT * FROM run;
  
```

STEP #4 – WITH RECURSIVE

```

1 pow(x,n) =
  let i0 = 0 in
  let p0 = 1 in
  run(i0,p0,x,n)

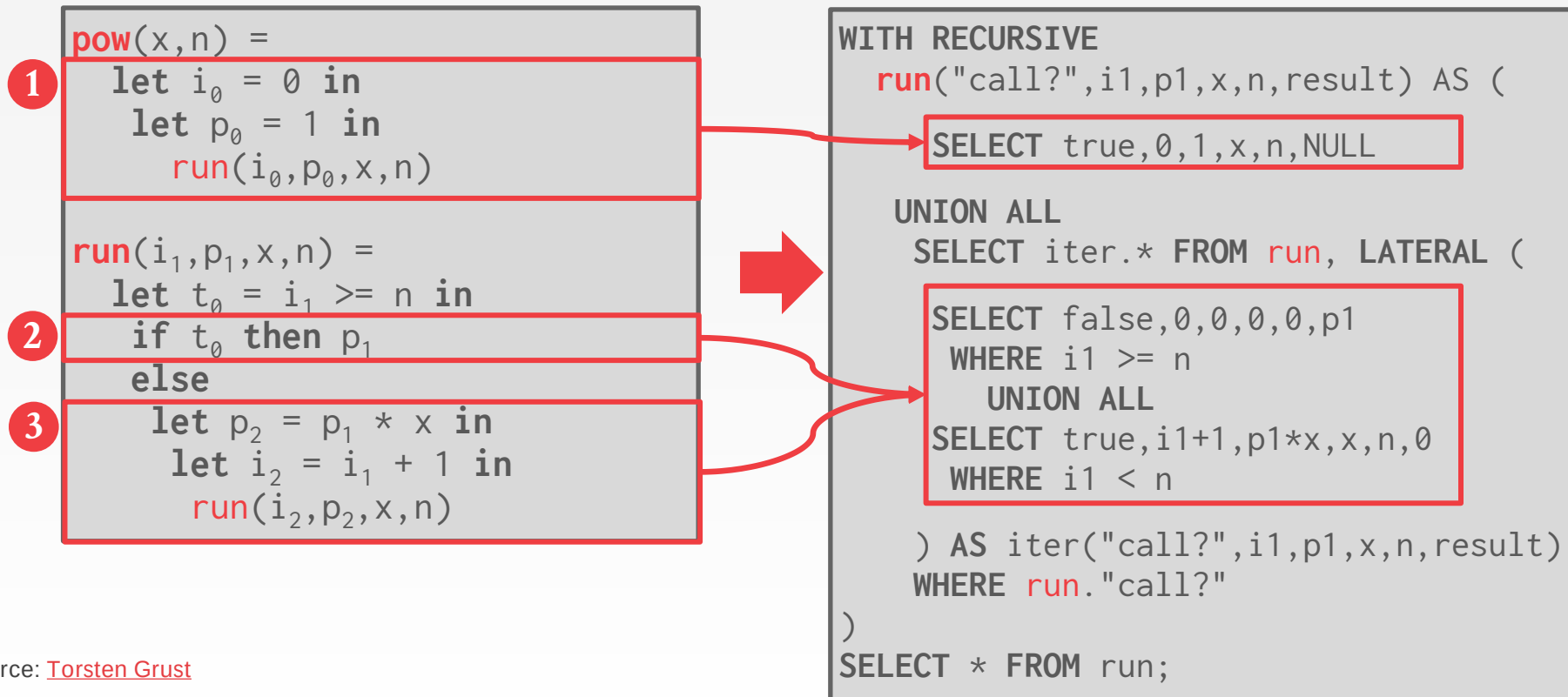
run(i1,p1,x,n) =
  let t0 = i1 >= n in
  if t0 then p1
  else
    let p2 = p1 * x in
    let i2 = i1 + 1 in
    run(i2,p2,x,n)
  
```



```

WITH RECURSIVE
  run("call?",i1,p1,x,n,result) AS (
    SELECT true,0,1,x,n,NULL
  UNION ALL
    SELECT iter.* FROM run, LATERAL (
      SELECT false,0,0,0,0,p1
      WHERE i1 >= n
      UNION ALL
      SELECT true,i1+1,p1*x,x,n,0
      WHERE i1 < n
    ) AS iter("call?",i1,p1,x,n,result)
    WHERE run."call?"
  )
SELECT * FROM run;
  
```

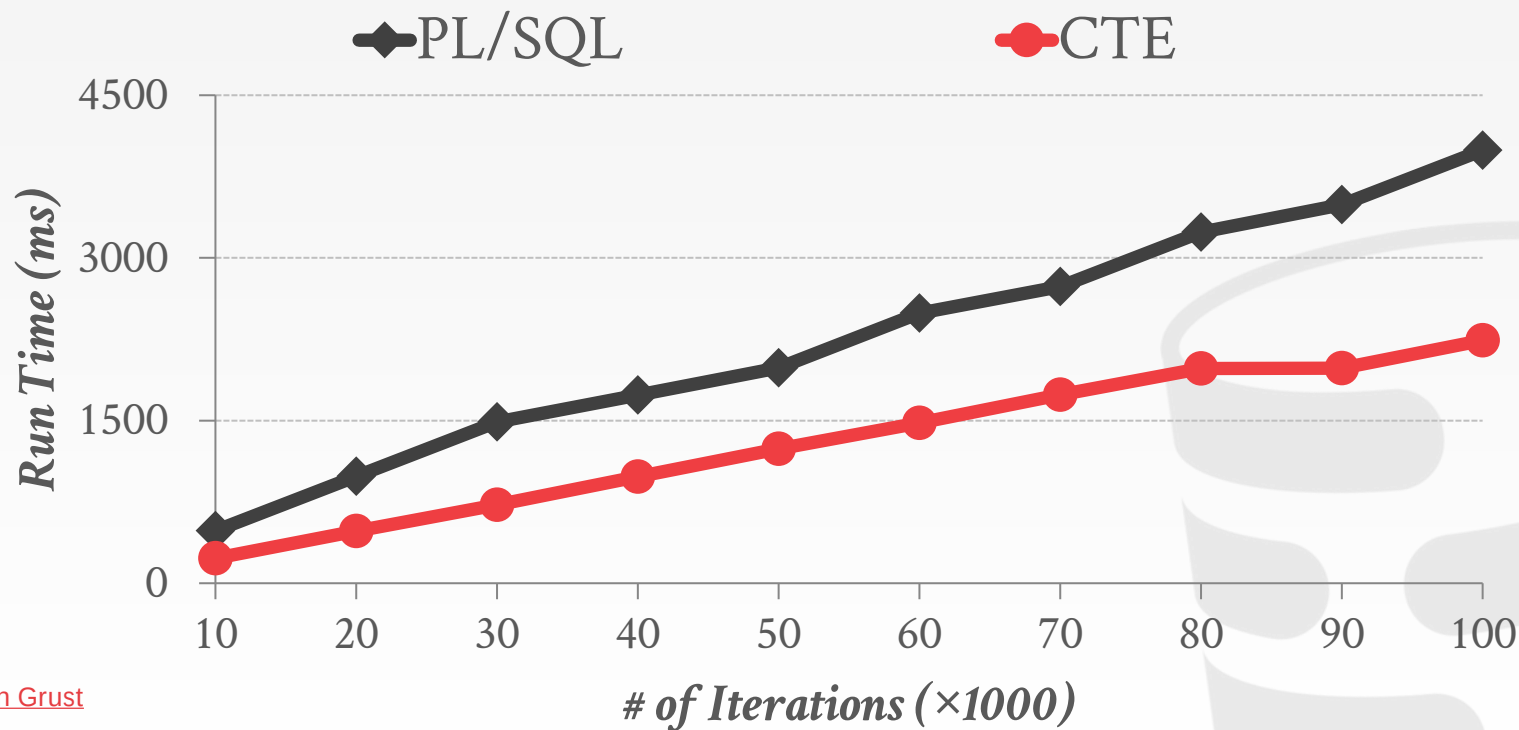

STEP #4 – WITH RECURSIVE



Source: [Torsten Grust](#)

UDFs-TO-CTEs EVALUATION

POW UDF on Postgres v11.3



Source: [Torsten Grust](#)

PARTING THOUGHTS

This is huge. You rarely get 500x speed up without either switching to a new DBMS or rewriting your application.

Another optimization approach is to compile the UDF into machine code.

→ This does not solve the optimizer's cost model problem.

NEXT CLASS

Last Lecture: Databases on New Hardware



CMU · 15-721 | Advanced Database Systems (2020)

15-721 (2021) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1wv411w7Ko>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-15-721>

内存数据库

并发控制

数据库压缩

哈希连接

超内存数据库

存储模型

调度规划

查询执行

索引

协议

网络

查询编译

查询优化

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程资料包页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击底部菜单栏
称为 AI 内容创作者？回复 [添砖加瓦]